Scheme Saathi: Roles and Work Plan

Team guide for Group G05 (Agentic AI Capstone, Project 1). Use it to decide who fits which role. Source: the capstone architecture and build guide dated 16 October 2026.

1. The project in one minute

- **Goal:** help a family find government welfare schemes they qualify for, explain why in Hindi, Marathi or English, and prepare an application that a person approves before submission.
- **Users:** a citizen and the Common Services Centre (CSC) operator helping them.
- **Rule 1:** the AI never decides eligibility. A tested rule engine (plain code) decides.
- **Rule 2:** text from outside (web pages, uploads, tool results) is data, never instructions.
- **Rule 3:** a person approves every real submission, tied to that exact form. Any edit voids the approval.
- **Signature challenge A:** turn scheme prose into JSON rules (field, operator, value, source clause), scored against 15 hand-coded gold schemes, improved with a Reflexion loop and human-reviewed lessons.
- **Signature challenge B:** ask the fewest questions (pick the question that settles most open verdicts) and give the same verdicts in all three languages.

2. Roles at a glance

#	Role	Core focus	Best fit for someone who...
1	Domain and product lead	Users, scheme choice, policies, personas, demo	knows public services or domain work, writes clearly, likes talking to people
2	Knowledge and memory engineer	Ingestion, search, reranker, memory	enjoys data pipelines, search, embeddings, databases
3	Agents and orchestration engineer	LangGraph, planner, agents, rule extractor, Reflexion	strong Python, comfortable with LLM prompts and state machines
4	Tools, control and security engineer	MCP servers, hooks, approval gate, guardrails, threat model	careful, security minded, likes APIs and contracts
5	Evaluation and red-team engineer	Personas, harness, judge, experiments, attacks	analytical, likes metrics, testing and breaking things
6	Deployment and operations engineer	FastAPI, UI, Docker, CI, Grafana, alerts	DevOps or backend experience, enjoys dashboards and automation

Everyone must be able to redraw the full architecture from memory and justify every box in the viva, and each person must explain their own part in depth.

3. Each role in detail

Role 1: Domain and product lead

Mission: Owns the problem, the users, the written policies and the content of the test cases. Talks to any domain expert and owns the demo script.

Responsibilities

- Define the user, the problem and the limits of the claims we make.
- Choose the central schemes and one state (Maharashtra fits the Pune, Hindi and Marathi example).
- Choose the 40 to 60 schemes and the 15 schemes to hand-code as gold rules. Record the URL and collection date for each.
- Write the hand-coded gold rules (with role 3 reviewing the format).
- Write persona content: 60 synthetic personas around rule boundaries (age 59 vs 60, income just over a limit).
- Write the written policies: what the system refuses, when a case goes to a human caseworker, what counts as official source.
- Check the held-out cases against the written policy (required separation rule).
- Own the demo script and the user-facing wording in all three languages.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• Scheme list with URL and date • Gold rules for 15 schemes • Persona content • Policy notes • Demo script	• Reads long documents carefully • Comfortable in Hindi and Marathi helps a lot • Clear writing, attention to detail	Roles 2, 3 and 5 need your outputs early. Start collection first.

Role 2: Knowledge and memory engineer

Mission: Builds everything that finds and remembers information: ingestion, search and the memory store with its policy.

Responsibilities

- Collect and parse scheme pages and PDFs, split into clauses, keep scheme, state, URL and date as metadata.
- Build hybrid search: BM25 plus multilingual vectors (for example bge-m3), merged with Reciprocal Rank Fusion, plus a reranker.
- Add one query technique, such as rewriting colloquial text (for example 'widow pension') into scheme language.
- Return passage IDs so answers can cite them.
- Measure Recall@5 on 30+ labelled queries, before and after one change. Keep honest numbers even if a change does not help.
- Build the memory store: pseudonymous household ID, consent flag, expiry, delete command, and a note on why each field is stored.
- Write the one-page memory policy (what is saved, when, who can read, how to correct or delete).
- Show a memory-on versus memory-off comparison.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• Ingestion pipeline • Hybrid index and reranker • Recall@5 report • Memory store • Memory policy page	• Python data work • Search or embeddings basics • Databases	Feeds the schemes-kb tool server (role 4) and the eligibility agent (role 3).

Role 3: Agents and orchestration engineer

Mission: Builds the brain: the graph, the plan, the four specialist agents, the rule extractor and the improvement loop.

Responsibilities

- Build the LangGraph graph: typed state (HouseholdProfile), conditional edges, one self-correction loop, persistent checkpointer.
- Show a plan object to the user and replan when a tool result contradicts it.
- Build the four agents: intake, eligibility analyst, document checker, application agent, with typed hand-off messages.
- Question selection: choose the next question by how many open verdicts it can settle. Measure questions per conversation.
- Make the same persona receive the same verdicts in Hindi, Marathi and English.
- Build the rule extractor (prose to JSON rules). Score it against the 15 gold schemes.
- Run the Reflexion loop on development schemes: write a short lesson for each error type, have a person review each lesson, freeze the playbook before the held-out run.
- Build the single-agent baseline (same tools and budget) for comparison.
- Demonstrate stop and resume after a crash mid-application.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• Graph diagram • Working agents • Rule extractor and accuracy • Reflexion playbook • Resume demo	• Strong Python • Prompt writing • Comfort with state machines	Needs the tool contracts (role 4), search (role 2) and gold rules (role 1).

Role 4: Tools, control and security engineer

Mission: Builds the tool servers and everything that keeps the system safe: hooks, approval gate, guardrails and the threat model.

Responsibilities

- Build four FastMCP servers with typed inputs and outputs, structured errors and a read or write mark on every tool: schemes-kb (search, get scheme, documents required), rule-engine (evaluate profile against scheme), doc-check (extract and compare fields) and mock-portal (draft, submit, status).
- Build the mock portal as a small FastAPI app with its own database. Only the application agent may use the write tools.
- Write one test per tool and publish the tool list with schemas.
- Build pre-tool hooks (permission and budget check) and post-tool hooks (trace with personal data scrubbed).
- Build the approval gate with interrupt(): approve, edit or reject. The approval token is tied to a fingerprint of the exact form, expires, and works once. Show the live demo where an edit invalidates an earlier approval.
- Build guardrails: mask Aadhaar and bank numbers before any model or trace, injection check, output check that every verdict cites a clause, no scam links, no leaked IDs. Send ID numbers only to mock-portal.submit.
- Allow only official sources to supply rules. Refuse requests to misstate income or age.
- Write the one-page threat model and the least-privilege table (which agent may call which tool).
- Route can't-tell verdicts and conflicting documents to a human caseworker.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• Four MCP servers and tests • Hooks • Approval gate and demo • Guardrails • Threat model page	• Careful and security minded • APIs and typed contracts • Likes edge cases	Needs the rule JSON schema and profile shape early. Provides tools to role 3 and defences that role 5 tests.

Role 5: Evaluation and red-team engineer

Mission: Proves the system works honestly: test cases, harness, judge, experiments and attacks.

Responsibilities

- Build a golden set of 60 personas with expected outcomes written before running the agent. Split about 20 development and at least 40 held-out, keeping related variants together. Store held-out cases where the agent cannot read them.
- Two teammates label each persona against every candidate scheme independently.
- Build the simulated citizen (a different model with a hidden profile card that answers only what it is asked).
- Check with code first: final portal state, cited clauses exist, forbidden actions not taken, tool use. Use an LLM judge only where code cannot decide.
- Calibrate the judge (different model family) on 20+ hand-labelled outputs and report Cohen's kappa.
- Run four experiments from config files: E1 fixed workflow vs one agent vs team; E2 defences off vs on; E3 before and after the playbook; E4 LLM deciding vs rule engine (count boundary errors) and planned vs fixed question order.
- Report task success, policy violations, attack success, false blocks, cost per case, median and 95th percentile latency, errors. Repeat the final held-out comparison three times and report the spread.
- Build the red-team suite of 30+ attacks: scam page, injection in scheme text, requests to inflate eligibility, attempts to read another household's memory, IDs leaking into traces.
- Write the 4 to 6 page evaluation report and the experiment log.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• Golden set • Harness and simulated citizen • Judge agreement • E1 to E4 results • Red-team suite • Evaluation report	• Analytical • Likes metrics and statistics • Enjoys testing and breaking things	Needs working agents (role 3) and tools (role 4). Must stay independent of the people tuning and defending the agent.

Role 6: Deployment and operations engineer

Mission: Makes it run reliably for everyone and visible: service, UI, containers, CI, tracing and dashboards.

Responsibilities

- Serve the app with FastAPI and streaming replies, with sessions. Build the chat UI (Streamlit, Chainlit or Gradio).
- Make 'docker compose up' start the app, the MCP servers, the vector store, Prometheus and Grafana.
- Keep prompts and model IDs in one versioned config.
- Run evaluation in CI (GitHub Actions) and fail below a threshold. Keep one failed run with an explanation.
- Set up tracing for every LLM and tool call with personal data masked (LangSmith or Langfuse).
- Build the Grafana dashboard: questions per conversation, can't-tell rate, waiting approvals, masked-ID hits, cost per conversation.
- Set up one working alert that fires when the can't-tell rate jumps (a sign of a corpus or extractor problem).

- Write the README so the whole system starts with one command on a clean machine.

Deliverables / evidence	Good fit if you are...	Depends on / feeds
• FastAPI and UI • Docker Compose • CI eval gate • Dashboard and alert • README	• Backend or DevOps experience • Docker and CI • Likes dashboards	Needs all other roles' services to be containerisable. Agree on ports and config early.

4. Work plan by phase

Dates come from the cohort guidelines, which we do not have here, so the plan is by phase. Map the phases to the real deadlines in the meeting.

Phase	Goal	Role 1	Role 2	Role 3	Role 4	Role 5	Role 6
0. Setup	Roles, repo, shared contracts	Pick state and schemes	Pick stack for search	Draft graph and state	Draft tool contracts	Plan persona format	Repo, compose stub, config
1. Foundations	Data and tools exist	Collect schemes, write gold rules	Ingest, index, Recall@5 baseline	Graph skeleton and checkpointer	MCP servers and tests, mock portal	Persona labelling, golden set	FastAPI shell, tracing
2. Core build	End-to-end happy path	Persona and policy review	Memory store and policy	Agents, planner, question selection, extractor	Hooks, approval gate, guardrails	Harness and simulated citizen	UI, compose runs all services
3. Hardening	Safe and measured	Check held-out vs policy	Retrieval tuning, multilingual	Reflexion loop, single-agent baseline	Threat model, PII masking checks	Red-team suite, judge calibration	Dashboard, alert, CI gate
4. Experiments	Honest numbers	Review lessons	Memory on vs off	Freeze playbook	Fix defence gaps	Run E1 to E4, three repeats	Cost and latency capture
5. Finish	Submit and present	Demo script and rehearse	Docs	Graph diagram	Final demos	Evaluation report	README and backup recording

5. Dependencies: freeze these early

- **Rule JSON schema:** field, operator, value, source clause (roles 1, 3, 4).
- **HouseholdProfile structure** and the verdict format: eligible, not eligible, can't tell, with deciding clause and missing fields (roles 3, 4).
- **MCP tool contracts:** names, inputs, outputs, errors (roles 3, 4).
- **Typed hand-off messages** between agents (role 3).
- **Persona file format** and the held-out folder location (roles 1, 5).
- **Service ports and config file** for Docker (roles 4, 6).

6. Rules that keep results honest

- The person tuning the agent must not write the held-out cases.
- The person building the defences must not write the attacks.
- Role 5 owns both the held-out cases and the attacks. Role 1 checks the held-out cases against the written policy.
- If one person holds roles 4 and 5, a teammate must write or review the attacks and the held-out cases, and the contribution log must say so.
- Keep gold answers out of prompts and tools. Never edit results by hand. If a change is made after seeing held-out results, treat those cases as development cases and say so.
- No API keys, private data or non-redistributable material in the repository.

7. Questions to settle in the meeting

- Who takes which role? Give each person a first and second choice, and name a backup for every role.

- Which state and which 40 to 60 schemes? Who collects them, and by when?
- Which 15 schemes become the hand-coded gold set?
- Who labels the personas (two independent labellers per persona) and who writes the attacks?
- Which models and whose OpenRouter budget? Agree on rate-limit care.
- When do we freeze the shared contracts, and how often do we sync?
- Where do we keep the contribution log, starting day one?

8. Deliverables checklist (whole team)

- Repository with a README that starts everything with one command.
- Our own redrawn architecture diagram with a numbered walkthrough.
- Threat model page, memory policy page, evaluation report (4 to 6 pages), experiment log, contribution log.
- Slides and a backup demo recording.
- Demo: (1) Hindi widow-pension request to an approved draft; (2) refusing to under-report income and ignoring a scam page; (3) 'Forget my data' taking effect.